

OOP in Embedded-Try zero-cost abstraction



How far can we push OOP into embedded systems — without paying for it?

The Premise

In systems programming, there is a common assumption:

"OOP and high-performance kernel development don't mix."

In many cases, this assumption is justified — typical OOP usage does introduce cost.

But what if the problem is not OOP itself?

What if the real issue is that we haven't been strict enough about how abstractions are built?

The Baseline: C Already Works

C dominates embedded systems for a simple reason:

- Direct hardware control
- Minimal runtime overhead
- Predictable execution behavior

It works.

If OOP is not used, there is no problem to solve — C is already sufficient.

So, introducing OOP is not a necessity. It is a choice.

And that choice comes with a constraint:

OOP is only justified when its performance is close enough to C.

Reclaiming OOP (Not Rejecting It)

Many embedded developers take a defensive stance toward OOP because common implementations introduce:

- Hidden runtime behavior
- Unpredictable cost
- Loss of determinism

PeltOS takes a different position:

OOP is not rejected — it is filtered.

Some aspects of OOP are clearly worth using:

- Structure and modularity
- Maintainability
- Expressive abstraction

But only under one condition:

They must satisfy the performance model.

More concretely:

Abstraction must not change how the hot path executes.

From Instruction Scheduling to Abstraction Design

In earlier work, I manually arranged instructions to better fit the CPU pipeline:

- Breaking dependency chains
- Filling latency gaps
- Reducing control flow

The goal was always:

Keep the pipeline full, avoid stalls.

Compilers already attempt this through instruction scheduling and reordering.

But they do not remove the root cause.

PeltOS applies the same idea at a different level:

I am no longer arranging instructions to fit the pipeline — I am designing abstractions so the pipeline does not need to adjust at all.

A Practical Starting Point: Meyers Singleton

In practice, I started with the well-established Meyers Singleton — for good reason.

It is widely regarded as one of the most efficient and elegant implementations.

```
class EventsQueue : public spinlock {
public:
    static EventsQueue& instance() {
        static EventsQueue evq;
        return evq;
    }
private:
    EventsQueue()
};
```

This pattern provides:

- Lazy initialization
- Thread safety
- Clean interface

Modern compilers optimize it very well.

Consider a hot-path call:

```
EventsQueue::instance().post();
```

what appears, at the C++ level, to be “two function calls” is usually not compiled into two visible runtime calls in the naive sense.

After inlining and optimization, the actual overhead is much smaller: in practice, what remains is typically the singleton guard check plus the object access path. Function-local statics do incur a hidden initialization check in embedded C++ discussions, and that check may stay in the hot path even when the singleton is already initialized.

A simplified assembly-like shape looks roughly like this: And it is already highly optimized.

```
mov    al, [guard_flag]
test   al, al
jne    hot_path          ; branch, usually predicted

call   init_function      ; cold path
mov    [guard_flag], 1

hot_path:
mov     rax, [instance_addr]
call    [rax + post_offset]
```

So, the result is already very close to what embedded developers would accept:

- The branch is highly predictable
- The initialization path is cold
- The hot path is already near C-level efficiency

Branch prediction exists precisely to reduce the control-flow penalty of such branches, especially in deeper pipelines.

That means the Meyers approach is not merely “usable” — for many cases, it is already good enough.

So, the question is not whether it qualifies.

It does.

The real question is: *Is there still room to optimize further?*

One More Step: Removing the Guard

In PeltOS, I explored that next step by separating initialization from access:

```
class EventsQueue : public spinlock {
public:
    static void init() noexcept {
        if (!ptr_) {
            ptr_ = new (storage_) EventsQueue();
        }
    }

    static inline EventsQueue& instance() noexcept {
        return *ptr_;
    }

private:
    static inline alignas(EventsQueue) unsigned char storage_[sizeof(EventsQueue)];
    static inline EventsQueue* ptr_ = nullptr;

    EventsQueue() = default;
};
```

Now consider the same call again:

```
EventsQueue::instance().post();
```

Once `EventsQueue::init()` has already been called, the hot path simplifies significantly.

A simplified assembly-like form becomes:

```
mov rax, [ptr_]
call [rax + post_offset]
```

The difference is small, but concrete:

- No singleton guard check
- No conditional branch
- No control-flow decision in the steady-state hot path

The hot path reduces to:

load → access → operate

At this point, the abstraction is no longer just “well optimized.”

It becomes structurally closer to a direct C-style access path.

What Actually Changed?

This is not about coding style.

It is a change in execution model.

- Meyers Singleton → very efficient, but still includes a guard check
- `init()` + `instance()` → removes that guard from the steady-state hot path

Or more precisely: *Instead of optimizing the branch, **this approach eliminates it from the hot path.***

Which leads to:

- Control dependency becomes simple data access
- Pipeline behavior becomes more explicit
- The abstraction gets even closer to C-level execution characteristics

The Cost of This Separation

Of course, this improvement is not free.

Separating `init()` from `instance()` introduces new responsibilities:

- `init()` must execute before `instance()` is ever used
- Lifetime becomes explicitly managed
- Automatic destruction is no longer part of the model

In other words:

We remove runtime cost by introducing stronger structural rules.

That is a real trade-off, and it should be stated plainly.

Design Philosophy of PeltOS

The starting point is simple:

C is the baseline.

It defines:

- the acceptable cost model
- the expected execution shape
- the level of control required

Introducing OOP is not about replacing C.

It is about recovering structure under a strict rule: *OOP must be indistinguishable from C in its performance footprint.*

If that condition is not met: *the abstraction is rejected*.

We are not using OOP for its own sake.

We use it only when it provides structure without introducing measurable overhead in the hot path.

Details as a First-Class Constraint

In PeltOS, detail is not an afterthought.

It is a design constraint.

Every abstraction is reduced to:

- its generated instructions
- its control flow shape
- its impact on the pipeline

If an abstraction hides cost, it is flawed.

Why This Matters in the AI Era

This emphasis on detail serves a larger purpose: to explicitly guide optimization.

AI is very good at:

- applying rules
- enforcing patterns
- scaling known solutions

But: *AI cannot infer instruction-level cost or pipeline behavior unless it is explicitly constrained*.

That means: *Optimization must be made explicit before it can be scaled*.

From Optimization to Rule

The transition from Meyers Singleton to `init()/instance()` is not just a micro-optimization.

It turns a local insight into a reusable rule:

- remove control dependency from hot paths
- prefer straight-line execution
- expose cost explicitly

Once defined, such rules can:

- guide future design
- be consistently applied

- be enforced by tools or AI

Summary

PeltOS is built on three constraints:

1. **C defines the cost model**
2. **OOP must comply with it**
3. **Details make that compliance enforceable**

Final Thought

PeltOS is not trying to use OOP — it is trying to rebuild it, under the performance contract defined by C.